1. Data Transfer Objects (DTOs)

Simple objects used to send data between client and server. They contain only needed fields and no business logic.

2. DTO Pattern

A design approach where internal models are converted into DTOs before sending data outside the application.

3. ModelMapper / MapStruct

Libraries used to convert Entities to DTOs and back automatically.

4. Create Resources

Using HTTP POST to add new data to the server.

5. Read Resources

Using HTTP GET to retrieve data from the server.

6. Update Resources

Using PUT (full update) or PATCH (partial update) to modify existing data.

7. Delete Resources

Using HTTP DELETE to remove data from the server.

8. Hypermedia Concept (HATEOAS)

An API design where responses include links to related actions or resources.

9. Links and Resources (REST Concept)

A resource is data identified by a URI; links connect related resources.

10. Spring HATEOAS

A Spring library that helps add hypermedia links to REST API responses.

11. Accept Header

A request header that tells the server what response format the client expects (like JSON or XML).

12. Content-Type Header

A header that specifies the format of the request or response body.

13. JSON vs XML

JSON is lightweight and easy to read; XML is more structured and commonly used in enterprise systems.

14. Actuator Endpoints & Health Checks

Features in Spring Boot that help monitor application health and performance.

15. Spring Security Basics

A framework that provides authentication (who you are) and authorization (what you can access).

16. JWT Authentication

A stateless authentication method where a signed token is used to verify users.

17. OAuth2

An authorization framework that allows secure access to resources using access tokens.

18. MockMvc

A Spring testing tool used to test controllers without running the full server.

19. @WebMvcTest

A Spring Boot annotation used to test only the controller layer.

20. OpenAPI / Swagger

A standard and toolset used to document and test REST APIs interactively.

21. SpringDoc

A library that automatically generates OpenAPI documentation in Spring Boot.

22. API Documentation Best Practices

Guidelines for writing clear, versioned, and example-based API documentation.

23. Monolith vs Microservices

Monolith is one large application; microservices split the app into smaller independent services.

24. Microservices Characteristics

Microservices are small, independent, loosely coupled services with their own databases.

25. Benefits and Challenges

Microservices improve scalability but increase system complexity.

26. Domain-Driven Design (DDD)

An approach that designs software based on business domains and bounded contexts.

27. Spring Cloud Overview

A set of tools that help build distributed microservices systems.

28. Service Discovery (Eureka)

A system where services register and discover each other dynamically.

29. Load Balancing (Ribbon / Spring Cloud LoadBalancer)

Distributes incoming requests across multiple service instances.

30. Circuit Breaker (Resilience4j)

Prevents repeated calls to failing services to avoid system crashes.

31. Microservices Security Challenges

Security issues caused by multiple services communicating over networks.

32. Service-to-Service Authentication

A way to ensure only authorized services communicate with each other.

33. CORS (Cross-Origin Resource Sharing)

A browser security feature that controls cross-domain requests.

34. Token-Based Security (Centralized)

Authentication using tokens issued by a central authorization server.

35. CSRF (Cross-Site Request Forgery)

An attack that tricks users into sending unwanted requests.

36. DDoS (Distributed Denial of Service)

An attack that overloads a server with massive traffic to make it unavailable.

37. OAuth2 Authorization Server

A server that authenticates users and issues access tokens.

38. Resource Server

A server that hosts protected APIs and validates access tokens.

39. SSO (Single Sign-On)

Allows users to log in once and access multiple applications.

40. RestTemplate

A synchronous HTTP client in Spring used to call REST APIs.

41. WebClient

A modern, non-blocking HTTP client for reactive applications.

42. Feign Client

A declarative REST client that simplifies calling other services.

43. Asynchronous Communication

Services communicate without waiting for immediate responses.

44. Spring Cloud Gateway

An API Gateway that routes requests to microservices.

45. Routing

The process of forwarding requests to the correct service.

46. Filters

Components that modify or inspect requests and responses.

47. Rate Limiting

Restricting the number of requests a client can make in a given time.

48. Retry Mechanisms

Automatically retrying failed requests.

49. Fallback Methods

Providing an alternative response when a service fails.

50. Bulkhead Pattern

Isolating resources to prevent failure in one part from affecting others.

51. Centralized Configuration

Storing application configuration in one central location.

52. Config Server

A server that provides configuration to multiple microservices.

53. Config Client

A service that fetches configuration from a Config Server.

54. @RefreshScope

Allows configuration properties to refresh without restarting the app.

55. Distributed Tracing (Sleuth, Zipkin)

Tracking requests across multiple services to monitor performance.

56. Logging Aggregation

Collecting logs from multiple services into one system.

57. Metrics Collection

Monitoring system performance like CPU, memory, and request time.

58. Defense in Depth

Using multiple layers of security for better protection.

59. API Security

Protecting APIs using authentication, validation, encryption, and monitoring.

60. Secrets Management

Securely storing sensitive information like passwords and API keys.

1. JPA Overview

JPA is a Java specification used to map Java objects to database tables. It reduces the need to write JDBC code.

2. Spring Data JPA Benefits

It simplifies database operations by providing built-in CRUD methods and automatic query generation.

3. Repository Pattern Setup

We create an interface extending JpaRepository, and Spring automatically provides the implementation.

4. Dependencies Configuration

Add JPA and database driver dependencies in pom.xml to enable database support in Spring Boot.

5. DataSource Configuration

DataSource connects the application to the database using URL, username, and password.

6. JPA Properties

Settings that control Hibernate behavior like schema creation and SQL logging.

7. @Entity

Marks a class as a database table in JPA.

8. @Table

Specifies the table name and details for an entity.

9. @Column

Maps a class field to a database column.

10. Primary Keys

A unique identifier for each record in a table, defined using @Id.

11. Relationships (@OneToMany, etc.)

Annotations used to define relationships between entities (like one-to-many).

12. Embedded Objects

Used to group related fields into one reusable class without creating a new table.

13. CrudRepository

Provides basic CRUD operations like save, find, and delete.

14. JpaRepository

Extends CrudRepository and adds pagination and sorting features.

15. PagingAndSortingRepository

Provides methods for pagination and sorting of data.

16. Custom Repositories

Used to define custom queries when default methods are not enough.

17. Save and Update

The save() method inserts or updates data depending on whether the ID exists.

18. Find Methods

Methods like findByName() automatically generate queries based on method names.

19. Delete Operations

Used to remove records using methods like deleteById().

20. Query Derivation

Spring creates queries automatically by reading repository method names.

21. @Query

Used to write custom JPQL or SQL queries manually.

22. Named Queries

Predefined queries written at the entity level and reused in repositories.

23. Native Queries

Raw SQL queries written directly inside repository methods.

24. Pageable

Used to fetch data in pages instead of loading all records at once.

25. Sort

Used to sort query results in ascending or descending order.

26. Page vs Slice

Page gives total count information; Slice only shows if the next page exists.

27. @CreatedDate

Automatically stores the date when an entity is created.

28. @LastModifiedDate

Automatically stores the date when an entity is updated.

29. @CreatedBy

Stores the user who created the record.

30. @LastModifiedBy

Stores the user who last modified the record.

31. Auditing Configuration

Enables automatic tracking of created and modified details in entities.

32. Code Quality Metrics

Measurements like coverage and complexity used to evaluate code health.

33. Technical Debt

Future problems caused by taking shortcuts in development.

34. Code Smells

Signs of poor coding practices that may cause maintainability issues.

35. SonarQube Dashboard

A tool dashboard that shows code quality issues and technical debt.

36. REST Principles

Guidelines for designing scalable web services using HTTP and stateless communication.

37. HTTP Methods

GET (read), POST (create), PUT (update), DELETE (remove), PATCH (partial update).

38. HTTP Status Codes

Numbers like 200, 404, 500 that indicate the result of an API request.

39. Spring Boot 3 Features

Supports Java 17+, Jakarta namespace, improved performance, and better observability.

40. @RestController

Used to create REST APIs that return JSON/XML directly.

41. @RequestMapping

Maps HTTP requests to controller methods.

42. @GetMapping

Handles HTTP GET requests to retrieve data.

43. @PostMapping

Handles HTTP POST requests to create new data.

44. @RequestBody

Converts JSON request data into a Java object.

45. @ResponseBody

Sends Java object data directly as JSON/XML in the response.

46. @PathVariable

Extracts values from the URL path.

47. @RequestParam

Extracts query parameters from the URL.

48. ResponseEntity

Represents the full HTTP response including body, headers, and status code.

1. Interface-based Projections

A way in Spring Data JPA to fetch only selected fields using interfaces instead of full entities. It improves performance and is mainly used for read-only data.

2. Class-based Projections

A method of fetching specific fields into a custom class (DTO) using a constructor. It is clear, explicit, and useful for complex data transformations.

3. Dynamic Projections

A flexible approach where the projection type (Entity or DTO) is chosen at runtime. It reduces duplicate repository methods.

4. Auto-configuration

A Spring Boot feature that automatically configures the application based on dependencies and properties. It reduces manual setup.

5. Properties Configuration

Application settings stored in application.properties or application.yml files. It allows easy management of environment-specific configurations.

6. Multiple DataSources

Using more than one database connection in a Spring Boot application. Each DataSource needs separate configuration.

7. Hibernate as JPA Provider

Hibernate is a tool that implements JPA and handles mapping between Java objects and database tables.

8. Entity Manager

The main JPA interface used to perform database operations like save, update, delete, and find entities.

9. Session Factory

A Hibernate component that creates Session objects to interact with the database. It is similar to EntityManagerFactory in JPA.

10. Caching (First-Level & Second-Level)

First-level cache works within a single transaction, while second-level cache stores data across multiple sessions to improve performance.

1. Spring Framework Overview

Spring is a lightweight framework used to build Java applications.

It provides features like Dependency Injection, security, and transaction management.

2. Spring Ecosystem

Spring Ecosystem is a collection of projects like Spring Boot, MVC, Data, Security, and Cloud.

Each project solves a specific problem in enterprise development.

3. Spring vs Java EE

Spring is lightweight and flexible, while Java EE is specification-based.

Spring Boot simplifies setup compared to traditional Java EE.

4. Introduction to Maven

Maven is a build and dependency management tool for Java projects.

It manages libraries and project lifecycle using pom.xml.

5. Maven Project Structure

Maven follows a standard folder structure like src/main/java.

This keeps projects organized and consistent.

6. Project Object Model (POM)

POM is the pom.xml file in Maven.

It contains dependencies, plugins, and build configuration.

7. Maven Build Lifecycle

Maven lifecycle has phases like compile, test, package, and install.

Running mvn install executes all phases up to install.

8. Spring Dependencies

Spring dependencies are libraries needed to use Spring features.

They are added in pom.xml and downloaded automatically.

9. Inversion of Control (IoC)

IoC means Spring creates and manages objects instead of the developer.

This reduces tight coupling between classes.

10. IoC Container Types

Spring has two containers: BeanFactory and ApplicationContext.

ApplicationContext is more advanced and commonly used.

11. ApplicationContext

ApplicationContext is an advanced Spring container.

It manages beans and supports AOP, events, and internationalization.

12. BeanFactory

BeanFactory is the basic Spring container.

It creates beans only when requested (lazy loading).

13. XML vs Annotation Configuration

XML defines beans in XML files.

Annotations like @Component and @Autowired are modern and easier.

14. Bean Scopes

Bean scope defines how many bean instances are created.

Common scopes are singleton and prototype.

15. Constructor Injection

Dependencies are injected through the constructor.

It ensures required dependencies are initialized.

16. @Autowired

@Autowired automatically injects a dependency.

It works by type by default.

17. @Qualifier

@Qualifier specifies which bean to inject when multiple exist.

It resolves ambiguity.

18. AOP

AOP separates cross-cutting concerns like logging and security.

It keeps business logic clean.

19. Cross-Cutting Concerns

These are features like logging and transactions used across modules.

They are separated using AOP.

20. Aspect, Advice, Pointcut

Aspect contains cross-cutting logic.

Advice defines what to do, and Pointcut defines where to apply it.

21. @Aspect

@Aspect marks a class as an AOP aspect.

It contains advice methods like @Before and @After.

22. @Before, @After, @Around

@Before runs before method execution.

@Around runs before and after and can control execution.

23. AOP Proxies

Spring uses proxy objects to apply aspects.

Proxies add extra behavior without changing original code.

24. Setter Injection

Dependencies are injected using setter methods.

Useful for optional dependencies.

25. Field Injection

Dependencies are injected directly into fields.

It is not recommended for production.

26. Best Injection Type

Constructor Injection is best.

It ensures immutability and better testability.

27. Spring MVC Architecture

Spring MVC follows Model-View-Controller pattern.

It separates data, UI, and business logic.

28. DispatcherServlet

DispatcherServlet is the front controller.

It handles all incoming HTTP requests.

29. Controllers

Controllers handle client requests.

They return a response or view.

30. ViewResolver

ViewResolver maps logical view names to actual files.

Example: “home” → home.jsp.

31. Model and View

Model holds data, and View displays it.

Controller connects both.

32. Spring ORM

Spring ORM integrates ORM frameworks like Hibernate.

It simplifies database operations.

33. Hibernate Integration

Hibernate maps Java objects to database tables.

Spring manages its configuration and transactions.

34. What is Spring Boot?

Spring Boot simplifies Spring application development.

It provides auto-configuration and embedded servers.

35. Spring Boot vs Spring

Spring provides core features.

Spring Boot simplifies configuration and deployment.

36. Auto-Configuration

Auto-configuration automatically configures beans based on dependencies.

It reduces manual setup.

37. Starter Dependencies

Starters are predefined dependency bundles.

Example: spring-boot-starter-web.

38. Spring Boot CLI

CLI allows running Spring apps from command line.

Used for quick prototyping.

39. application.properties / application.yml

Used to configure Spring Boot applications.

They define server port, DB config, etc.

40. Reactive Programming

Reactive programming handles asynchronous data streams.

It is non-blocking and scalable.

41. Project Reactor

Project Reactor is a reactive library used in WebFlux.

It provides Mono and Flux types.

42. Mono and Flux

Mono returns 0 or 1 value.

Flux returns multiple values.

43. WebFlux vs MVC

MVC is blocking.

WebFlux is non-blocking and reactive.

44. Reactive Controllers

They return Mono or Flux.

Used in WebFlux applications.

45. Reactive Data Access

Uses non-blocking database drivers like R2DBC.

Supports fully reactive applications.

46. @SpringBootTest

Used for integration testing.

It loads the full application context.

47. @MockBean

Creates a mock bean in Spring context.

Used to replace real beans during testing.

48. Testing Controllers

Use @WebMvcTest and MockMvc.

It tests endpoints without loading full context.

49. Testing Services

Use JUnit and Mockito.

Repositories are mocked to test business logic.

50. HTTP Client Mocking

Used to mock external API calls in tests.

Prevents calling real third-party services.

51. Database Mocking

Used to test logic without real database.

Often uses Mockito or in-memory H2 databas